

Manuale operativo

Guida alla pubblicazione e alla manutenzione del sito

claudiocammarano.com · Eleventy 3 · Giugno 2026

1. Panoramica del sistema

Il sito è un generatore statico costruito su **Eleventy 3** con template Nunjucks. Non esiste un database: ogni contenuto è un file Markdown con frontmatter YAML. Eleventy legge quei file, applica i template e produce HTML puro in una cartella `_site/`, che viene poi servita da Netlify.

Questo approccio ha un vantaggio cruciale: il sito non può «cadere» per problemi di database o di server applicativo. Il costo è che ogni modifica richiede un build e un deploy. In pratica il ciclo è sempre lo stesso: si modifica un file sorgente, si fa *git push*, Netlify rileva il push e aggiorna il sito in produzione automaticamente (in genere in meno di due minuti).

Struttura delle cartelle principali

I contenuti editoriali stanno tutti in **src/**. Le tre sottocartelle che si usano quotidianamente sono:

Cartella	Contenuto
<code>src/writings/</code>	Saggi originali di Claudio
<code>src/curated/</code>	Link esterni con commento editoriale
<code>src/learning/</code>	Note di apprendimento e report
<code>src/_data/</code>	Dati globali (tassonomie, descrizioni)
<code>src/css/</code>	Unico file di stile (style.css)
<code>src/images/</code>	Immagini degli articoli

I comandi fondamentali

Per lavorare sul sito è necessario avere Node.js installato. Tutti i comandi si eseguono dalla cartella del progetto:

```
cd ~/Documents/PROJECTS/claudio-cammarano-site npm start # avvia il server locale su
http://localhost:8080 npm run build # build di produzione git add -A && git commit -m
'messaggio' && git push # deploy
```

Il server locale con `npm start` ricarica il browser in automatico a ogni modifica ai file sorgente. È lo strumento principale per verificare il risultato prima di pubblicare.

2. Pubblicare un saggio (writing)

I saggi sono i contenuti principali del sito: testi originali, analitici, spesso lunghi, con struttura argomentativa. Ogni saggio è un file Markdown in **src/writings/** il cui nome segue la convenzione *AAAA-MM-GG-slug-in-italiano.md*.

Lo slug si ricava dal titolo: tutto minuscolo, spazi sostituiti da trattini, accenti rimossi (è→e, à→a, ò→o, ù→u, ì→i), caratteri speciali eliminati, massimo sei o sette parole significative. Esempio: *"L'ombra del futuro"* diventa **2026-04-15-lombra-del-futuro.md**.

Il frontmatter

Ogni file inizia con un blocco YAML delimitato da `---` che contiene i metadati della pagina. Eleventy legge questi metadati e li usa per generare titolo, breadcrumb, tag, Open Graph e molto altro.

```
--- layout: layouts/article.njk title: "Titolo completo del saggio" date: 2026-06-19
description: "1-2 frasi di pitch editoriale, non riassunto." category: ["AI",
"Geopolitica"] lang: "■■ Italiano" tags: [writings] ---
```

Campi facoltativi:

og_image: `"/images/nome.avif"`

Immagine usata per l'anteprima sui social e come hero dell'articolo. Formati supportati: avif, jpg, png. File in `src/images/`.

series: `"Nome serie, I"`

Indica che il saggio è parte di una serie. Il formato è esatto: nome, virgola, spazio, numero romano. Esempio: "Teoria dei giochi e ordine internazionale, II". Le pagine `/serie/` si generano automaticamente.

english_version: `"https://..."`

URL della versione inglese (di solito su Substack). Il template mostra automaticamente un link alla versione tradotta.

serie_totale_prevista: `3`

Numero intero. Indica quanti episodi sono previsti nella serie. Appare nella pagina `/serie/` come "2 di 3 previsti".

Le categorie

Il campo *category* determina il cluster tematico dell'articolo, che compare nel breadcrumb e nella pagina `/temi/`. Qui sotto i valori «storici», usati finora dai saggi (maiuscola inclusa). L'elenco completo degli Argomenti del sito — molto più ampio, alimentato anche dai curated — è in `clusters.js` (capitolo 5): un saggio può in linea di principio usarne uno qualsiasi. Si possono combinare categorie di cluster diversi se il pezzo è genuinamente trasversale.

Cluster	Valori validi
Epistemologia & AI	AI · Epistemologia · Filosofia · Scrittura · Vibe Coding · Claude
Geopolitica & potere	Democrazia · Geopolitica · Medio Oriente · Mediterraneo Regimi politici · Scenario Planning · Teoria dei giochi
Editoria & comunicazione	Comunicazione · Dual Use · Formazione · Scienze della Comunicazione

Italia & istituzioni

Bologna · Humanities · Istituzioni · Italia · Politica

Struttura del corpo

Il corpo dell'articolo è Markdown standard con alcune estensioni HTML per gli elementi visivi. La struttura tipica prevede un'immagine hero opzionale all'inizio, seguita dal testo diviso in sezioni con intestazioni H2.

Il grassetto (**testo**) viene usato per asserzioni concettuali chiave, sempre come frasi intere e mai per singole parole. Il corsivo (*testo*) è riservato a titoli di libri e riviste. I link sono sempre inline e accompagnati da contesto sufficiente a capire perché il link è rilevante.

Elementi speciali disponibili:

```
{%- kicker "ETICHETTA BREVE" %} {%- pullquote "Citazione lunga in evidenza con bordo sinistro." %} {%- figure "/images/nome.jpg", "Didascalia" %} <!-- Info-box con sfondo grigio e bordo blu: --> <div class="info-box"> <p>Contenuto della nota.</p> </div>
```

Se il saggio contiene formule matematiche in LaTeX, inserire il blocco MathJax subito dopo la riga di chiusura del frontmatter e prima del testo. Le formule inline si scrivono tra dollari singoli ($\$formula\$$), quelle a display tra doppi dollari ($\$\$formula\$\$$).

La bibliografia

I saggi di tipo accademico o che citano fonti primarie chiudono con una sezione **## Bibliografia essenziale**. Il formato è autore-data stile APA: cognome, iniziale (anno). *Titolo in corsivo*. Città: Editore. I titoli di articoli vanno tra virgolette, i nomi di riviste in corsivo.

Collegare i concetti — passaggio manuale, non automatico

A differenza dei curated, per i saggi il collegamento ai concetti non passa dal frontmatter del file: vive in **src/_data/conceptsIndex.js**. **Salvare il file .md non basta**. Se l'articolo cita un concetto già presente in tassonomia, va aggiunta una voce `{ title: "Titolo articolo", url: "/writings/slug-articolo/" }` all'array `articles` della voce corrispondente. Se introduce un concetto nuovo, va creata una nuova voce (procedimento al capitolo 4).

La skill pubblica-articolo non esegue questo passaggio: genera solo il file markdown del saggio, non tocca conceptsIndex.js. Il collegamento ai concetti va fatto a mano, articolo per articolo.

Checklist completa per pubblicare un saggio:

```
1. Salvare il file in src/writings/ 2. Aprire src/_data/conceptsIndex.js e aggiornare le voci dei concetti citati (esistenti o nuovi) 3. npm run build in locale, e leggere la console: se un concetto non è registrato, ora il build lo segnala (capitolo 4) 4. git push Non esiste un comando separato di "aggiornamento semantico": indici, pagine /concetti/, chip e grafo si rigenerano da soli al build, una volta che i dati sono corretti.
```

3. Pubblicare un curated

I curated sono segnalazioni di articoli, post o risorse esterne accompagnate da un commento editoriale. A differenza dei saggi, non hanno una pagina propria sul sito: vengono mostrati in elenco nella pagina `/curated/` con un link esterno. Il file va in **src/curated/** e il nome segue la stessa convenzione data-slug.

Il frontmatter

```
--- title: "Titolo originale dell'articolo esterno" external_url:
"https://url-completo.com/articolo" source: "Nome Autore / Nome Pubblicazione" date:
2026-06-19 description: "2-4 frasi in prima persona con punto di vista editoriale."
tags: [curated, ai, geopolitica] concepts: ["Anthropic", "allineamento AI"] ---
```

Il campo **description** è il più importante dei curated. Non è un riassunto neutro dell'articolo: è il motivo per cui vale la pena leggerlo, visto dall'angolazione del sito. Spiega cosa porta di nuovo, come si incastra con i temi ricorrenti, perché Claudio lo ha segnalato. Il tono è in prima persona.

Il campo **tags** inizia sempre con *curated*, seguito da uno o tre tag tematici in minuscolo libero. Il campo **concepts** contiene nomi dalla tassonomia dei concetti (capitolo 4). I nomi sono case-sensitive: vanno copiati esattamente come compaiono in quella lista. Se nessun concetto si applica, si lascia *concepts: []*.

I tags ora contano come Argomenti — ma solo se «riconosciuti»

I tag liberi in *tags*: vengono tradotti in un Argomento vero — visibile in `/indice/`, `/temi/` e con una pagina `/tag/...` propria — tramite la mappa in **src/_data/curatedTagAliases.js**. Se il tag che scrivi (case-insensitive) è già una chiave in quella mappa, l'articolo conta per l'Argomento corrispondente: scrivi *ai* minuscolo, l'articolo compare sotto l'Argomento *AI*.

*Se il tag **non** è in quella mappa, resta puramente decorativo: non genera nessuna pagina, non compare in nessun indice. Non è un errore, ma è facile non accorgersene. La procedura per aggiungere un tag nuovo alla mappa è al capitolo 5.*

4. Navigazione semantica

Il sito ha tre livelli di navigazione concettuale, dal più ampio al più specifico: **Temi** (5 cluster, /temi/), **Argomenti** (i singoli tag, circa 49, ognuno con una pagina /tag/slug/) e **Concetti** (persone, teorie, testi, istituzioni, luoghi e paesi citati con peso argomentativo, ognuno con una pagina /concetti/slug/). L'**indice analitico** (/indice/) li mostra tutti e tre insieme, proprio per chiarire su quale livello ci si trova in ogni momento.

Concetti: come funziona

Ogni concetto ha una pagina propria all'indirizzo **/concetti/slug/** con: una riga di chip per i «concetti vicini» (concetti che condividono almeno un articolo); la lista degli articoli collegati; e una **finestra sul grafo globale** — una porzione ritagliata del grafo completo, centrata sul concetto, con una minimappa che mostra dove si trova rispetto all'insieme. Un link «Vedi mappa completa →» apre **/mappa/** centrata sullo stesso nodo.

Il grafo è uno solo: viene calcolato una volta a build time da *graphLayout.js* (layout a forze scritto in JS puro, nessuna libreria esterna) ed esposto come JSON statico su */graph-data.json*. Ogni pagina concetto e */mappa/* leggono quel file e ne mostrano porzioni diverse — non sono grafi separati. **È scope-limitato a Concetti e articoli**: Temi e Argomenti non ci entrano, per scelta. Visibile solo da schermi $\geq 900\text{px}$; su mobile resta la sola lista testuale, come prima dell'introduzione del grafo.

I chip con i nomi dei concetti compaiono automaticamente in fondo a ogni saggio e sotto la descrizione di ogni curated. Nell'indice analitico ogni nome è un link cliccabile. Tutto questo avviene in automatico a ogni build: non c'è nulla da configurare per ogni singolo articolo.

Il flusso dei dati

Per i saggi (*writings*): l'Argomento viene dal campo *category*: nel frontmatter; il Concetto va collegato a mano in **src/_data/conceptsIndex.js**, nell'array *articles* di ogni voce. Per i curated: l'Argomento viene dal campo *tags*:, tradotto tramite *curatedTagAliases.js* (capitolo 3); il Concetto viene dal campo *concepts*:, che deve corrispondere a una voce già presente in *conceptsIndex.js*.

Non esiste un comando per «aggiornare l'indice semantico»: è lo stesso identico git push (o npm run build) con cui si pubblica il sito. Se i dati a monte sono corretti, il build successivo rigenera automaticamente indici, pagine concetto, Temi, Argomenti e grafo. Non c'è nulla da richiamare a parte.

Aggiungere un nuovo concetto

Se un nuovo saggio introduce un personaggio, una teoria o un'istituzione che merita di entrare nell'indice, il procedimento è il seguente:

1. Aprire **src/_data/conceptsIndex.js**.
2. Aggiungere una nuova voce nell'array con i campi *name*, *type* e *articles*. Il campo *name* è l'identificatore usato ovunque nel sistema: va scelto con cura e non cambiato in seguito.
3. Inserire nell'array *articles* tutti i saggi già pubblicati che citano il concetto, con titolo e URL.

4. Fare il build: la pagina /concetti/slug/ appare automaticamente.
5. Per i curated futuri, sarà sufficiente aggiungere il nome al campo *concepts*: del frontmatter.

*I tipi validi per il campo type sono: **persona, teoria, testo, istituzione, luogo, paese**. La scelta del tipo determina in quale sezione dell'indice compare il concetto. **Ogni nome di stato è «paese», non «luogo»** — anche quando nel linguaggio comune lo chiameremmo «un luogo» (Giappone, Germania, Iran...). «luogo» è riservato a città, regioni e snodi geografici specifici che non sono uno stato (Bologna, Beirut, Taiwan come territorio conteso).*

Il controllo automatico — niente più concetti persi in silenzio

Per anni il sistema ha scartato **senza nessun avviso** qualsiasi nome in *concepts*: di un curated che non corrispondesse a una voce già presente in `conceptsIndex.js`. Risultato: articoli pubblicati con concetti che restavano invisibili nell'indice e nel grafo, senza che nessuno se ne accorgesse — a volte per mesi.

Ora il build segnala il problema in console, a ogni npm run build o npm start, con il nome del file e del concetto mancante. Lo stesso messaggio compare anche nei log di deploy di Netlify — quindi resta visibile anche pubblicando solo via git push, senza build locale.

```
■■ CONCETTI NON REGISTRATI (ignorati nell'indice/grafico) – aggiungili a  
src/_data/conceptsIndex.js: ./src/curated/2026-06-21-esempio.md -> "Nome Concetto"
```

Se lo vedi: apri `conceptsIndex.js`, aggiungi la voce mancante, rifai il build. L'avviso scompare quando tutto è registrato — non blocca il deploy, è solo un promemoria.

5. Temi e Argomenti (clusters.js)

`src/_data/clusters.js` è la tassonomia dei Temi: un oggetto che mappa ogni nome di Tema a un array di Argomenti. Cinque Temi in tutto — il quinto, «Economia & lavoro», è nato dai curated: nessuno dei 4 Temi originari aveva una casa naturale per quei contenuti.

```
module.exports = { "Epistemologia & AI": ["Filosofia", "AI", "Vibe Coding", ...],  
  "Geopolitica & potere": ["Geopolitica", "Difesa", "Europa", ...], "Editoria &  
  comunicazione": ["Comunicazione", "Media", ...], "Italia & istituzioni": ["Italia",  
  "Bologna", "Femminismo", ...], "Economia & lavoro": ["Economia", "Lavoro"] };
```

Per aggiungere un Argomento a un Tema, basta inserirlo nell'array del Tema corrispondente: `/temi/`, `/indice/` e il breadcrumb si aggiornano al prossimo build.

Per i writings

Basta scrivere il nome esatto in *category*: (capitolo 2). Se il nome non esiste ancora come Argomento, va prima aggiunto a `clusters.js`, altrimenti l'Argomento non avrà un Tema e non apparirà in `/temi/`.

Per i curated — un passaggio in più

I tag dei curated (*tags*: nel frontmatter) sono scritti liberi e minuscoli, non nel formato canonico degli Argomenti. Per contare davvero come Argomento, ogni tag deve prima essere tradotto in `src/_data/curatedTagAliases.js`:

```
module.exports = { "ai": "AI", "surveillance": "Sorveglianza", "tuo-tag-nuovo": "Nome  
  Canonico" };
```

Se un tag non è in quella mappa, resta puramente decorativo: non genera nessuna pagina `/tag/...`, non compare in nessun indice. Non è un errore — ma è facile non accorgersene. Quando un curated introduce un tema nuovo: decidi il nome canonico (Title Case, in italiano per coerenza — «Sorveglianza», non «surveillance»), aggiungi la riga in `curatedTagAliases.js`, poi aggiungi quel nome all'array del cluster giusto in `clusters.js`. Una descrizione in `tagDescriptions.json` è facoltativa ma consigliata.

Se il tema in realtà è un'entità — una persona, un paese, un'istituzione — probabilmente appartiene ai Concetti (capitolo 4), non agli Argomenti: è il caso tipico dei nomi di paese, che vanno nel campo `concepts`, non in `tags`.

Le descrizioni dei Temi vivono in `temiDescriptions.json`, quelle degli Argomenti in `tagDescriptions.json` — entrambe facoltative, entrambe scritte in prima persona e ancorate ai contenuti reali del sito, non enciclopediche.

6. Gestire le serie

Una serie è un gruppo di saggi collegati da un filo tematico comune, pubblicati in sequenza. Il sito gestisce le serie automaticamente: le pagine `/serie/` e `/serie/nome-serie/` si generano senza intervento manuale.

Per dichiarare che un saggio fa parte di una serie, aggiungere il campo *series* al frontmatter con il formato esatto: *"Nome serie, I"* (nome, virgola, spazio, numero romano). Il template riconosce il numero romano, lo usa per l'ordinamento e mostra il nome della serie senza il suffisso numerico.

Se la serie ha una lunghezza pianificata, aggiungere *serie_totale_prevista*: 3 (o il numero corretto). Questo valore appare nella pagina listing come «2 di 3 previsti».

Per aggiungere una descrizione della serie che appare nella pagina `/serie/`, inserire una voce in `src/_data/seriesDescriptions.json` con il nome base della serie (senza il numero romano) come chiave.

7. File da non toccare senza capire

Il sito ha alcuni file centrali la cui modifica errata può bloccare il build o rompere il layout di tutte le pagine. Qui sotto una guida rapida su cosa fa ciascuno e qual è il rischio.

File	Funzione	Rischio
<code>.eleventy.js</code>	Configurazione del generatore: collections, filters, layouts, shortcodes, etc.	Un errore qui, si rompe tutto, l'intero build.
<code>src/_includes/layouts/article.html</code>	Template usato da tutti i saggi.	Un errore rompe la resa di tutti gli articoli.
<code>src/_includes/layouts/base.html</code>	Template base del sito: header, footer, meta tag.	Un errore rompe ogni singola pagina del sito.
<code>src/css/style.css</code>	Unico file CSS (~3800 righe). Contiene tutte le modifiche di design.	Modifiche di design possono alterare il layout ovunque.
<code>src/_data/conceptsIndex.js</code>	Tassonomia dei concetti: fonte di verità per indici e pagine concetti.	Non è paginabile, se si erra rompono le pagine concetto.
<code>src/_data/clusters.js</code>	Tassonomia Temi/Argomenti: usata da <code>/temi/</code> , <code>/indici/</code> , <code>/tema-tutti/</code> .	Un errore qui fa sparire da Temi e indice, pur restando funzionante come pagina <code>/tag/</code> .
<code>src/_data/curatedTagAliases.js</code>	Trasforma i tag liberi dei curated in Argomenti canonici.	Se viene cambiato qui «spegne» silenziosamente quell'Argomento i curated già pubblicati.
<code>src/_data/graphLayout.js</code>	Calcola le posizioni del grafo concetti a build.	Si rigenera da solo da <code>conceptsIndex.js</code> e <code>clusters.js</code> : non richiede modifiche manuali.

8. Troubleshooting

Quasi tutti i problemi che si incontrano rientrano in poche categorie. Qui di seguito i più comuni con la diagnosi e la soluzione.

Un concetto/articolo appena pubblicato non appare da nessuna parte

Prima cosa da controllare: lanciare `npm run build` e leggere la console. Se il concetto è citato in un `concept`: di un `curated` ma non registrato, ora compare un avviso ■■ CONCETTI NON REGISTRATI con il nome del file e del concetto mancante (capitolo 4). Risolve la maggior parte dei casi.

Cerco un Concetto-paese e non lo trovo in «Luoghi»

I nomi di stato sono paese, non luogo: controllare nella sezione «Paesi» dell'indice analitico (capitolo 4).

Il build fallisce con errore Nunjucks

C'è un tag `{% %}` non chiuso o una variabile non definita in un file `.njk`. L'errore indica il file e la riga. Aprire quel file e correggere la sintassi.

Il build fallisce con errore YAML

Il frontmatter di un file `.md` ha un problema di sintassi: un apice non chiuso, un'indentazione errata o un carattere speciale non escaped. Verificare il file segnalato nell'errore.

Una pagina /concetti/ non appare pur risultando registrata

Il nome nel campo `concepts`: del file `curated` non corrisponde esattamente (maiuscole comprese) a un name in `conceptsIndex.js`. Se non corrisponde, ricompare nell'avviso di build (capitolo 4).

I chip concetti non appaiono in un saggio

L'URL del saggio non è presente nell'array `articles` della voce corrispondente in `conceptsIndex.js`. Aggiungere la voce con titolo e URL esatti.

Un tag di un curated non genera nessuna pagina /tag/...

Non è in `curatedTagAliases.js` (capitolo 3): è decorativo, non un Argomento. Aggiungerlo alla mappa se merita di diventarlo (capitolo 5).

Il grafo non compare

Il grafo (pagine concetto e /mappa/) è visibile solo su schermi con larghezza $\geq 900\text{px}$. Su mobile è sostituito dalla lista articoli, invariata. Verificare su desktop.

Netlify build fallisce ma il build locale funziona

Di solito dipende da una versione diversa di Node o da un file che non è stato committato (git status mostra file non tracciati). Controllare il log su `app.netlify.com` → Deploy. Un avviso di concetti non registrati nei log non è un errore: non blocca il deploy.

Le immagini non appaiono

Verificare che il file sia in `src/images/` e che il path nel frontmatter (`og_image`) o nel markup (`article-hero`) sia esatto, inclusa l'estensione (`.avif`, `.jpg`, `.png`).

9. La skill di pubblicazione

Per semplificare la pubblicazione è disponibile una skill per Claude chiamata **pubblica-articolo**, installabile da Impostazioni → Funzionalità → Skills. Una volta attiva, è sufficiente incollare un testo grezzo o un link con una nota e chiedere a Claude di pubblicarlo: la skill genera il file .md completo con frontmatter corretto, tassonomia applicata, filename e struttura pronti per essere salvati.

La skill conosce l'intera tassonomia (categorie, concetti, cluster), le convenzioni di stile editoriale del sito, i template frontmatter per entrambi i tipi di contenuto e le regole di naming dei file. L'output è un file copiabile direttamente in src/writings/ o src/curated/ senza modifiche.

***Limite attuale:** per i saggi, la skill non aggiorna conceptsIndex.js. Genera solo il file markdown del saggio: il collegamento ai concetti (capitolo 2) resta un passaggio manuale, anche quando si usa la skill. Per i curated invece non c'è questo limite, perché il campo concepts: sta nel frontmatter che la skill genera già completo.*

Se si aggiungono nuovi concetti a conceptsIndex.js, aggiornare la skill tramite skill-creator in modo che li conosca. La skill non si aggiorna da sola.

Stesso limite, non ancora risolto, vale per i tag dei curated: la skill scrive tags: liberi nel frontmatter, ma non sa se quel tag è già tradotto in un Argomento da curatedTagAliases.js (capitolo 5). Un tag nuovo coniato dalla skill resta decorativo finché non lo si aggiunge a mano alla mappa.

*Questo documento va aggiornato ogni volta che si introduce una modifica strutturale al sito: nuovi campi frontmatter, nuovi template, modifiche al sistema di navigazione semantica. La versione Markdown di riferimento è **MANUALE.md** nella root del progetto.*
